

lab7_leila_kawanishi_277185

Leila Kawanishi

SQLite - Parte II

Benilton Carvalho, Guilherme Ludwig

O SQLite é uma biblioteca embutida em um processo único que é autônomo, não necessita de servidor e exige zero de configuração. O código é de domínio público e seu uso é gratuito em qualquer tipo de utilização. Trata-se de um sistema SQL completo, capaz de utilizar múltiplas tabelas, índices, gatilhos e visões. Ele funciona, virtualmente, em qualquer plataforma (incluindo móveis) e pesa cerca de 600KB.

Objetivos

Ao fim deste laboratório, você deverá ser capaz de:

- Conectar-se a um banco de dados do tipo SQLite utilizando o R como interface;
- Explorar quais são as tabelas disponíveis no referido banco de dados;
- Identificar quais são as colunas de uma dada tabela existente no banco de dados;
- Realizar pesquisas simples;
- Extrair registros do banco de dados e armazená-las em objetos do R;
- Realizar algumas pesquisas complexas, utilizando WHERE, INNER JOIN.
- Criar novas tabelas no banco de dados.

Fonte do problema: Estes dados são de uma iniciativa de publicidade de desempenho de alunos e professores nas Universidades Americanas. A UW-Madison disponibilizou os seus dados online (no Office of the Registrar: <https://registrar.wisc.edu/>), e alguém trouxe os dados para uma competição no Kaggle (<https://www.kaggle.com/Madgrades/uw-madison-courses>). Nesta atividades, examinaremos disciplinas oferecidas no assunto de Estatística pela referida Universidade. O esquema do banco de dados é apresentado na figura abaixo.

Atividade

Baixe o arquivo uwmadison.sqlite3. Conecte-se a ele usando o pacote RSQLite, armazenando a conexão em uma variável conn.

```
library(RSQLite)
conn <- dbConnect(SQLite(), '../archive/database.sqlite3')
dbListTables(conn)
```

```
[1] "course_offerings"  "courses"           "grade_distributions"
[4] "instructors"       "rooms"              "schedules"
[7] "sections"          "subject_memberships" "subjects"
[10] "teachings"
```

Quem são e quantos são os professores que lecionaram disciplinas cujo tópico era estatística (subjects.abbreviation='STAT');

```
dbListFields(conn, "subjects")
```

```
[1] "code"          "name"          "abbreviation"
```

```
dbListFields(conn, "subject_memberships")
```

```
[1] "subject_code"      "course_offering_uuid"
```

```
dbListFields(conn, "course_offerings")
```

```
[1] "uuid"          "course_uuid" "term_code"   "name"
```

```
dbListFields(conn, "sections")
```

```
[1] "uuid"          "course_offering_uuid" "section_type"
[4] "number"        "room_uuid"          "schedule_uuid"
```

```
dbListFields(conn, "teachings")
```

```
[1] "instructor_id" "section_uuid"
```

```
dbListFields(conn, "instructors")
```

```
[1] "id"  "name"
```

```
teachers <- dbGetQuery(conn,
  'SELECT DISTINCT INSTRUCTORS.NAME FROM SUBJECTS
  INNER JOIN SUBJECT_MEMBERSHIPS
    ON SUBJECTS.CODE = SUBJECT_MEMBERSHIPS.SUBJECT_CODE
  INNER JOIN COURSE_OFFERINGS
    ON SUBJECT_MEMBERSHIPS.COURSE_OFFERING_UUID = COURSE_OFFERINGS.UUID
  INNER JOIN SECTIONS
    ON COURSE_OFFERINGS.UUID = SECTIONS.COURSE_OFFERING_UUID
  INNER JOIN TEACHINGS
    ON SECTIONS.UUID = TEACHINGS.SECTION_UUID
  INNER JOIN INSTRUCTORS
    ON TEACHINGS.INSTRUCTOR_ID = INSTRUCTORS.ID
  WHERE SUBJECTS.ABBREVIATION = "STAT"')
teachers
```

	name
1	MINJING TAO
2	DONALD PORTER
3	SHENG WANG
4	KUNLING HUANG
5	DONG XIA
6	JENNIFER NGUYEN
7	SEULKEE YUN
8	DUZHE WANG
9	YONGFENG WU
10	JARED BROWN
11	YUCHEN ZHOU
12	VICTOR LUO
13	GUN WOONG PARK
14	BROOK LUERS
15	DUY NGUYEN
16	JOHN GILLET
17	QUOC TRAN
18	YAoyao XU
19	JIE ZHANG
20	LONG PHAN
21	WESLEY CHANG

22	TONG LI
23	SHANE HUBLER
24	FAN YANG
25	JU HEE CHO
26	JEEA CHOI
27	SHIXUE LIU
28	WEI-YIN LOH
29	BIN DAI
30	XIPEI YANG
31	DOUGLAS M. BATES
32	MICHAEL GEORGE ILTIS
33	LIAM JOHNSTON
34	CRYSTAL CHEN
35	YIFAN MEI
36	RUI CHEN
37	JILI WANG
38	LUWAN ZHANG
39	GINA BENNINGER
40	JIE SONG
41	XIWEN MA
42	HAODA FU
43	DEYUAN JIANG
44	JINGJIANG PENG
45	ERICA LEE DEADMAN
46	AKICHIKA OZEKI
47	MENG SONG
48	HAO TENG
49	DONGGYU KIM
50	COLIN LONGHURST
51	FREDERICK BOEHM
52	YONGSU LEE
53	GONZALO CONTADOR
54	HAO CHEN
55	XIN ZHANG
56	BO YANG
57	RUNGANG HAN
58	SEAN KENT
59	DEBRAJ DAS
60	YI LI
61	CLAIRE BOBST
62	CHAN PARK
63	SIJING LI
64	SHENGJI JIA

65	YUCHANG WU
66	ZHUANG WU
67	XIAOMAO LI
68	HYEBIN SONG
69	YOURAN QI
70	YOUNG MIN PARK
71	BIN ZHANG
72	QIURONG CUI
73	SEHO PARK
74	SHENG ZHANG
75	HUIKUN ZHANG
76	ALYSSA DIGILIO
77	YUQING XU
78	STEPHEN BERG
79	PEIGEN ZHOU
80	YAN CHEN
81	QIONG ZHANG
82	YOUNGDEOK HWANG
83	HAN CHEN
84	NICHOLAS STEPHEN KEULER
85	ZHENGXIAO WU
86	YUJIN CHUNG
87	WENWEN ZHANG
88	PERLA REYES
89	QI TANG
90	CHIA-CHIEH LIN
91	JIALE XU
92	YANG ZHAO
93	XINWEI DENG
94	MING XIE
95	JINGCI MENG
96	ISMOR FISCHER
97	JIAJIE CHEN
98	ZUOFENG SHANG
99	HAO ZHENG
100	XU XU
101	BIN ZHU
102	QUEFENG LI
103	YU QIU LIU
104	JUN LI
105	FAN GAO
106	VICTORIA MANSFIELD
107	ANQI SHI

108	KYLE HEBERT
109	XIUYU MA
110	CHENLIANG XU
111	CHANHAN HSU
112	YUANZHI LI
113	KAM-WAH TSUI
114	XINYU SONG
115	TIMOTHY IDOWU
116	XIAO NIE
117	CHEN CHENG
118	ANDREW LESLIE
119	YAOGUO XIE
120	KEVIN PACKARD
121	YI LIU
122	WENZHI CAO
123	JOSEPH DEUTSCH
124	LAN LUO
125	RYAN ZEA
126	ROBERT WARDROP
127	SANGBUM CHOI
128	XIN LI
129	SEUNGBONG HAN
130	ZHIGUO XIAO
131	RICHARD A. JOHNSON
132	RUI TANG
133	IAN BRANSTAD RILEY
134	BO HUANG
135	HEATHER MARIE BRAZEAU
136	XU HE
137	GARY HOWARD SCHROEDER
138	JUN ZHANG
139	CHENXI LI
140	SONA Z SWANSON
141	YUAN JIANG
142	JUNHEE HAN
143	LILUN DU
144	XIRAN WANG
145	ZIFENG ZHAO
146	CUIZE HAN
147	YING ZHANG
148	JOHN DAVIS
149	CHEN JING
150	JINGLAN LI

151	YI CHAI
152	THU LE
153	ALAN HUANG
154	HAOYANG FAN
155	BOWEN HU
156	TZU HSIANG HUNG
157	TUN LEE NG
158	ABIGAIL BENZINE
159	ALEXANDER COVINGTON
160	CHELSEY GREEN
161	MICHAEL KUTZLER
162	LILI ZHENG
163	REBECCA POST
164	LU YANG
165	RUOSI GUO
166	XUN ZHAO
167	MIN NIU
168	JIAN WU
169	WEI ZHENG
170	XINJIE HE
171	JIE WEI
172	CLAUDIA SOLIS LEMUS
173	XUEYAO CHEN
174	MANJUSHA KANCHARLA
175	MITCHELL PAUKNER
176	XIAOWU DAI
177	CHENGNING ZHANG
178	GINA OH
179	LUXI CAO
180	SONG WANG
181	LAM HO
182	TIEN VO
183	SHIZHEN WANG
184	SOHEIL SADEGHI
185	BINGYING XIE
186	GENG LI
187	JENNIFER BIRSTLER
188	MARIA KAMENETSKY
189	KYLE HERBET
190	JIANCHANG HU
191	TRAM TA
192	YUAN WANG
193	KAZUHIKO SHINKI

194	SIJIAN WANG
195	CHIEN-WEI CHEN
196	BEHZAD AALIPUR
197	DEREK BEAN
198	BEN ADAM HAALAND
199	ELOISA D CHAVAS
200	YALI WANG
201	VARSHA KULKARNI
202	SCOTT JOSEPH HETZEL
203	MIN JUNG LEE
204	FANG FANG
205	KRISTEN CYFFKA
206	TING-LI LIN
207	BRET LARGET
208	KATHERINE GOODE
209	ERIK NORDHEIM
210	NING FAN
211	ANRU ZHANG
212	GARVESH RASKUTTI
213	KRISHNAKUMAR BALASUBRAMANIAN
214	YAZHEN WANG
215	CHAOYANG YU
216	YILIN ZHANG
217	ZHENGJUN ZHANG
218	XIUFENG SHAO
219	YUAN LI
220	BRET HANLON
221	LILI LAN
222	CECILE ANE
223	ZHANG CHUNMING
224	LEI XU
225	MUHONG GAO
226	AUGUST JENSEN
227	XIAO GUO
228	THOMAS G. KURTZ
229	NICHOLAS HENDERSON
230	CHENSHENG KUANG
231	TAERI UHM
232	NORBERT BINKIEWICZ
233	KARL ROHE
234	JUNGWON MUN
235	JOSEP GINEBRA
236	TAO YU

237	XINXIN YU
238	HAO ZHOU
239	LIE XIONG
240	QIAN ZHIGUANG
241	SHANG WU
242	SHAN LU
243	STEPHEN AARON STANHOPE
244	LANCINE KONATE
245	GUILHERME VIEIRA NUNES LUDWIG
246	PAUL SAVARIAPPAN
247	CUICUI QI
248	KJELL DOKSUM
249	DEREK NORTON
250	KEEGAN KORTHAUER
251	KAILEI CHEN
252	THEVAASIINEN CHANDERENG
253	JOHN KANE
254	SOKOL VAKO
255	QING LI
256	YOUNG LEE
257	HYUNSEUNG KANG
258	BEHZAD AALIPUR HAFSHEJANI
259	SHULEI WANG
260	ZHONGJIE YU
261	LIN QI
262	JIWEI ZHAO
263	SUNDUZ KELES
264	SHUYUN YE
265	HUI WANG
266	KEVIN HASEGAWA ENG
267	XIAODAN WEI
268	MARIA ROJO
269	RUIFENG XU
270	YONGJOON KIM
271	NELLIE LAUGHLIN
272	MICHAEL HOGAN
273	KARL BROMAN
274	REBECCA KOSCIK
275	RONGJUN ZHU
276	BI CHENG WU
277	YUNONG LIN
278	ZHIGENG GENG
279	XIAOPING FENG

280	SHUANG HUANG
281	GUANNAN SUN
282	JEE YEON KIM
283	GREGORY SHINAULT
284	TONGHAI YANG
285	TIMO SEPPALAINEN
286	BENEDEK VALKO
287	DAVID GRIFFEATH
288	DAVID ANDERSON
289	PHILIP WOOD
290	FLORIAN BERTRAND
291	JASON RICHARD SWANSON
292	ALEXANDER KISELEV
293	SAMUEL STECHMANN
294	WAI TONG FAN
295	JUN YIN
296	STEFFEN LEMPP
297	RUIFANG SONG
298	JAMES D KUELBS
299	ANATOLE BECK
300	DONGHYUN LEE
301	ANDREJ ZLATOS
302	DANIELE CAPPELLETTI
303	MIHAELA IFRIM
304	SEBASTIEN ROCH
305	SCOTT HOTTOVY
306	JONATHON PETERSON
307	STEPHEN WAINGER
308	ALEXANDER FISH
309	GREGORIO MORENO-FLORES
310	SUKHENDU MEHROTRA
311	DIETRICH UHLENBROCK
312	LEV BORISOV
313	RICHARD A BRUALDI
314	ARNOLD MILLER
315	MATTHEW BALLARD
316	ALBRECHT KLEMM
317	PAUL M TERWILLIGER
318	STEVEN SAM
319	KEN ONO
320	ROBERT HARRON
321	YANNAN QIU
322	JOHN WILTSHIRE-GORDON

323	KYUNGMAK KIM
324	CHRISTOPHER WAGNER
325	PAUL RATHOUZ
326	STEPHEN WRIGHT
327	BENJAMIN RECHT
328	MICHAEL FERRIS
329	JESSE THOMAS HOLZER
330	HUILIN HU
331	ALBERTO DEL PIA
332	ROBERT R MEYER
333	SHI JIN
334	MARY LINDSTROM
335	ADIN-CRISTIAN ANDREI
336	MICHAEL LIOU
337	CHAOQUN MEI
338	CHRISTINA M. KENDZIORSKI
339	RONALD GANGNON
340	ROBERT WAYNE GREEN
341	COLE COOK
342	MURRAY CLAYTON
343	QI JIANG
344	JUN ZHU
345	ANNE BRUCKNER
346	TING YE
347	JUN SHAO
348	HUAIBAO FENG
349	ZIJIAN NI
350	THOMAS COOK
351	RICHARD J. CHAPPELL
352	MOO K CHUNG
353	MICHAEL NEWTON
354	BRIAN YANDELL
355	MICHELLE HARRIS
356	NORMAN DRAPER
357	YU MENGANG
358	MING YUAN
359	FANGFANG WANG
360	JUNHO LEE
361	GRACE WAHBA
362	JASON P FINE
363	MICHAEL RENE KOSOROK
364	YAJUAN SI
365	LU MAO

```

366             DAVID DEMETS
367             JAMES ANDERSON
368             EDWARD ERKER
369             SHUAI CHEN
370             MARI PALTA
371             GUANHUA CHEN
372 SRIKANTHMADHAVAN ARAVAMUTHAN
373             NATALIA DE LEON GATTI
374             GUILHERME ROSA
375             YANBING ZHENG
376             CHRISTINE SORKNESS
377             MARIAN R FISHER

```

O GPA americano é definido numa escala de 0 a 4, em que A = 4, AB = 3.5, B = 3, BC = 2.5, C = 2, D = 1 e F = 0. Determinando a nota média de cada oferecimento pela ponderação da quantidade de alunos em cada extrato com os valores numéricos de cada conceito, indique (no que se referente a disciplinas no assunto de estatística):

```
dbListFields(conn, "grade_distributions")
```

```

[1] "course_offering_uuid" "section_number"    "n_count"
[4] "c_count"             "f_count"           "cr_count"
[7] "p_count"             "nw_count"          "other_count"
[10] "bc_count"            "u_count"           "d_count"
[13] "ab_count"            "a_count"           "s_count"
[16] "i_count"             "b_count"           "nr_count"

```

```
dbListFields(conn, "courses")
```

```
[1] "uuid"  "name"  "number"
```

```

gpa_stat <- dbGetQuery(
  conn,
  '
  SELECT
    course_offerings.uuid AS course_offering_uuid,
    course_offerings.name,
    course_offerings.course_uuid,
    gpa.GPA

```

```

FROM course_offerings

INNER JOIN subject_memberships
  ON course_offerings.uuid =
     subject_memberships.course_offering_uuid

INNER JOIN subjects
  ON subject_memberships.subject_code =
     subjects.code

INNER JOIN
(
  SELECT
    course_offering_uuid,

    (
      4 * SUM(a_count) +
      3.5 * SUM(ab_count) +
      3 * SUM(b_count) +
      2.5 * SUM(bc_count) +
      2 * SUM(c_count) +
      1 * SUM(d_count)
    )
    /
    (
      SUM(a_count) +
      SUM(ab_count) +
      SUM(b_count) +
      SUM(bc_count) +
      SUM(c_count) +
      SUM(d_count) +
      SUM(f_count)
    ) AS GPA

  FROM grade_distributions

  GROUP BY course_offering_uuid
) AS gpa

  ON course_offerings.uuid = gpa.course_offering_uuid

WHERE subjects.abbreviation = "STAT"

```

```
)
```

```
head(gpa_stat)
```

	course_offering_uuid	name
1	0a00ed96-3293-322f-ba23-039f67ce2fa7	Intro Stats for Engineers
2	8f3800fe-8d3f-392d-84a9-9509e888f6ac	Intro Stats for Engineers
3	03a155a1-32c6-3411-b388-9e59cea81c2c	Intro Stats for Engineers
4	bade8d5a-13d4-34ff-a1df-7a3f6fadd33f	Intro Stats for Engineers
5	fa4ccd19-cc15-3ca6-b69f-54fdface67d5	Intro Stats for Engineers
6	86d269e4-12ca-3aec-b630-fc94439893aa	Intro Stats for Engineers

	course_uuid	GPA
1	b013a134-06a6-31a9-9e46-128d03b4700e	3.008876
2	b013a134-06a6-31a9-9e46-128d03b4700e	2.915179
3	b013a134-06a6-31a9-9e46-128d03b4700e	2.871875
4	b013a134-06a6-31a9-9e46-128d03b4700e	3.352941
5	b013a134-06a6-31a9-9e46-128d03b4700e	3.565789
6	b013a134-06a6-31a9-9e46-128d03b4700e	3.009804

```
nrow(gpa_stat)
```

```
[1] 746
```

```
summary(gpa_stat$GPA)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NAs
2.441	3.167	3.461	3.430	3.692	4.000	83

```
professor_gpa <- dbGetQuery(  
  conn,  
  '  
  SELECT  
    instructor_offerings.instructor_id,  
    instructors.name,  
    AVG(instructor_offerings.GPA) AS mean_GPA  
  
  FROM  
    (  
    SELECT  
      instructor_offerings.instructor_id,  
      instructor_offerings.name,  
      AVG(instructor_offerings.GPA) AS mean_GPA  
    FROM  
      instructor_offerings  
    GROUP BY  
      instructor_offerings.instructor_id,  
      instructor_offerings.name  
    ) AS instructor_offerings  
  JOIN  
    instructors  
  ON  
    instructor_offerings.instructor_id = instructors.instructor_id  
  )
```

```

SELECT DISTINCT
    teachings.instructor_id,
    gpa_stat.course_offering_uuid,
    gpa_stat.GPA

FROM
(
    SELECT
        course_offering_uuid,

        (
            4 * SUM(a_count) +
            3.5 * SUM(ab_count) +
            3 * SUM(b_count) +
            2.5 * SUM(bc_count) +
            2 * SUM(c_count) +
            1 * SUM(d_count)
        )
        /
        (
            SUM(a_count) +
            SUM(ab_count) +
            SUM(b_count) +
            SUM(bc_count) +
            SUM(c_count) +
            SUM(d_count) +
            SUM(f_count)
        ) AS GPA

    FROM grade_distributions

    GROUP BY course_offering_uuid
) AS gpa_stat

INNER JOIN sections
    ON gpa_stat.course_offering_uuid =
        sections.course_offering_uuid

INNER JOIN teachings
    ON sections.uuid = teachings.section_uuid

INNER JOIN subject_memberships

```

```

        ON gpa_stat.course_offering_uuid =
            subject_memberships.course_offering_uuid

    INNER JOIN subjects
        ON subject_memberships.subject_code = subjects.code

    WHERE subjects.abbreviation = "STAT"
) AS instructor_offerings

INNER JOIN instructors
    ON instructor_offerings.instructor_id = instructors.id

GROUP BY
    instructor_offerings.instructor_id,
    instructors.name

HAVING mean_GPA IS NOT NULL

ORDER BY mean_GPA
'
)

```

a. Quem é o professor mais difícil? (menor GPA medio)

```
head(professor_gpa)
```

	instructor_id	name	mean_GPA
1	2601633	JAMES D KUELBS	2.607692
2	1634418	JONATHON PETERSON	2.776923
3	4194670	XIUFENG SHAO	2.778846
4	2601051	STEPHEN WAINGER	2.810833
5	5012817	KAILEI CHEN	2.810976
6	4680082	ALEXANDER FISH	2.813333

Respostas: o professor mais difícil é James D Kuelbs com GPA medio de 2.607692.

b. Quem é o professor mais fácil?

```
tail(professor_gpa)
```

	instructor_id	name	mean_GPA
369	3077702	MING YUAN	3.924561
370	3010828	YU MENGANG	3.927040
371	2600488	GRACE WAHBA	3.972642
372	3884290	SRIKANTHMADHAVAN ARAVAMUTHAN	4.000000
373	4272783	GUANHUA CHEN	4.000000
374	4426932	YAJUAN SI	4.000000

Resposta: O professores mais fáceis são Srikanthmadhavan Aravamuthan, Guanhua Chen e Tajuan Si, os três com GPA médio de 4.0.

```
course_gpa <- dbGetQuery(
  conn,
  '
  SELECT
    courses.uuid,
    courses.name,
    courses.number,
    AVG(gpa_stat.GPA) AS mean_GPA

  FROM
  (
    SELECT
      course_offering_uuid,

      (
        4 * SUM(a_count) +
        3.5 * SUM(ab_count) +
        3 * SUM(b_count) +
        2.5 * SUM(bc_count) +
        2 * SUM(c_count) +
        1 * SUM(d_count)
      )

    /

    (
      SUM(a_count) +
      SUM(ab_count) +
      SUM(b_count) +
      SUM(bc_count) +
      SUM(c_count) +
      SUM(d_count) +
      SUM(f_count)
    )
  )
  )
```

```

        ) AS GPA

FROM grade_distributions

GROUP BY course_offering_uuid
) AS gpa_stat

INNER JOIN course_offerings
ON gpa_stat.course_offering_uuid =
    course_offerings.uuid

INNER JOIN courses
ON course_offerings.course_uuid =
    courses.uuid

INNER JOIN subject_memberships
ON course_offerings.uuid =
    subject_memberships.course_offering_uuid

INNER JOIN subjects
ON subject_memberships.subject_code =
    subjects.code

WHERE subjects.abbreviation = "STAT"

GROUP BY courses.uuid, courses.name, courses.number

HAVING mean_GPA IS NOT NULL

ORDER BY mean_GPA
'
)

```

c. Qual é a disciplina mais difícil? (menor GPA)

```
names(course_gpa)
```

```
[1] "uuid"      "name"      "number"    "mean_GPA"
```

```
head(course_gpa)
```

```

                                uuid
1 3ef3ba60-426e-39ad-a61f-17d381b193d6
2 4c05436f-19b3-3f06-b99a-c28d268997f5
3 2eacb08c-4327-3ff0-aa76-deb43ec562c4
4 6bf60bd9-e21c-3a90-8848-e5552e136fd4
5 b013a134-06a6-31a9-9e46-128d03b4700e
6 f31fb5a4-74ce-33c5-b532-e5c7d4d2a92a

                                name number mean_GPA
1      Introduction to the Theory of Probability    431 2.878070
2                        Linear Programming Methods    525 2.957360
3 Introduction to Probability and Mathematical Statistics I    309 3.033830
4                        Introduction to Stochastic Processes    632 3.052279
5      Introductory Statistics for Engineers    224 3.073185
6      Introduction to Statistical Methods    301 3.088102

```

Resposta: A disciplina mais difícil é Introduction to the Theory of Probability com GPA médio de 2.87807

d. Qual é a disciplina mais fácil?

```
tail(course_gpa)
```

```

                                uuid
65 4f8beae6-3d74-37be-b5d7-3c6c8614ee27
66 e3a35c02-af4a-3d0b-bcc9-59a2e054a374
67 126a81bd-34e1-35f8-82f4-5edb70c3ae8b
68 61e7c0a0-e62b-37fc-bed5-db70c90c4f92
69 74fb97c9-9e98-3e8a-b0d9-66123b5a8e33
70 99d2f325-3f56-397f-8c71-d4a2490a8e03

                                name number mean_GPA
65      Estimation of Functions from Data    860 3.977381
66      Experimental Design I    803 3.982051
67 Nonparametric Statistics and Machine Learning Methods    841 4.000000
68      Sample Survey Theory and Method    811 4.000000
69      Data Science Practicum    628 4.000000
70      Empir Proc&Semiparmtrc Infernc    834 4.000000

```

Resposta: E as disciplinas mais fáceis são Nonparametric Statistics and Machine Learning Methods, Sample Survey Theory and Method, Data Science Practicum. Empir Proc&Semi-paramtrc Infernc com GAP médio de 4.0 Desconecte do banco de dados.

```
dbDisconnect(conn)
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
```

```
paste(  
  "Arquivo compilado em:",  
  format(Sys.time(), "%d/%m/%Y às %H:%M:%S")  
)
```

```
[1] "Arquivo compilado em: 17/09/2026 às 13:08:07"
```